MDPI

Article

Short-Response Collapse in Countdown RLVR: A Matched Study of PPO, GRPO, and Asymmetric Clipping

Yongsheng Lian [1]

[1] Department of Mechanical Engineering, University of Louisville, Louisville, KY 40292, USA; yongsheng.lian@louisville.edu

Abstract

Reinforcement learning with verifiable rewards (RLVR) can raise an in-domain score while degrading the behavior that score is intended to represent. We test this risk by fine-tuning Qwen2.5-1.5B-Instruct on Countdown. We conduct nine 300-step runs: PPO, symmetric-clip GRPO, and asymmetric-clip DAPO-style training, with paired seeds 7–9 and matched data, reward, rollout count, actor updates, optimizer settings, and response limit. Early short-response collapse occurred in 1/3 PPO runs, 1/3 GRPO runs, and 3/3 asymmetric-clip runs. Across GSM8K, MATH, BBH, and MMLU-Pro, mean flexible accuracy under standard prompts remained near the base model (49.0%): 49.0% for PPO, 49.2% for GRPO, and 48.9% for DAPO-style training. Under the Countdown-style think prompt, the corresponding means were 43.4%, 32.8%, and 20.0%, exposing prompt-conditional damage. Strict scores rose without flexible-accuracy gains; a prompt-only control likewise raised base-model strict GSM8K accuracy by 25.2 points while lowering flexible accuracy by 5.2 points. A separate 22-run configuration screen shows collapse is frequent but configuration- and run-dependent. These results do not establish a universal algorithm ranking; they show that matched multi-seed training, flexible scoring, and prompt-robust evaluation are necessary to detect reward-compatible collapse.

Keywords: reinforcement learning with verifiable rewards; large language models; PPO; GRPO; training collapse; prompt robustness

1. Introduction

Reinforcement learning with verifiable rewards (RLVR) has become a prominent approach to eliciting mathematical and symbolic reasoning from large language models (LLMs). Systems such as OpenAI o1 [1] and DeepSeek-R1 [2] demonstrate that post-training can substantially change generated reasoning behavior. However, higher task reward does not by itself establish that a model has acquired broadly transferable reasoning capability. An apparent gain can instead arise from better adherence to the required final-answer format, a redistribution of probability toward solutions already available in the base model, or selection of an unusually successful training seed. Community reproductions such as TinyZero use Countdown as a small-model R1-Zero-style training task [22]; this motivates auditing whether its apparent reasoning gains are stable and transferable.

A variety of policy-optimization methods have been proposed for LLM post-training. Proximal Policy Optimization (PPO) uses a learned critic and clipped policy updates [3]. Group Relative Policy Optimization (GRPO) replaces the critic with advantages normalized over responses to the same prompt [4]. Decoupled Clip and Dynamic sAmpling Policy Optimization (DAPO) modifies clipping and loss aggregation and uses dynamic

Received:
Accepted:
Published:

Copyright: © 2026 by the author. Submitted to *AI* for possible open access publication under the terms and conditions of the Creative Commons Attribution (CC BY) license.

sampling to retain prompts whose response groups contain reward variation [5]. These methods are usually reported with different models, datasets, reward functions, rollout budgets, and optimization schedules, so comparisons between their published headline scores do not isolate algorithmic effects.

This single-model, single-domain study uses Qwen2.5-1.5B-Instruct and Countdown to ask four focused questions. First, when training exposure and optimization settings are matched, how often do PPO, symmetric-clip GRPO, and asymmetric-clip group-relative training collapse across paired seeds? Second, does the operational collapse label predict degraded behavior, or merely shorter answers? Third, do downstream changes survive both flexible scoring and a change of prompt format? Fourth, how do purpose-specific configuration, regularization, and evaluation controls qualify the matched comparison?

The matched experiment gives a simple central result. PPO and symmetric-clip GRPO each collapsed in one of three seeds, whereas asymmetric-clip DAPO-style training collapsed in all three. The damage was not apparent from standard-prompt flexible accuracy averaged over four downstream benchmarks, which remained approximately 49% for every fine-tuned model group and for the base model. It became clear under the Countdown-style think prompt: collapsed GRPO seed 9 averaged 18.2% versus 39.8% for the two non-collapsed GRPO seeds, and the three collapsed DAPO-style seeds averaged 20.0%. Thus, the failure is prompt-conditional rather than a uniform loss of benchmark accuracy. A separate 22-run configuration screen (16 collapses) and a prompt-only format control show that run configuration and the required final-answer format can strongly change the apparent conclusion.

Relative to the preliminary preprint [13], this article adds the matched nine-run comparison, the multi-run configuration screen, flexible rescoring, two-prompt evaluation, a prompt-only format control, pass@k analysis, and reproducibility manifests. The matched runs address controlled differences among training methods; the other experiment groups address configuration stability, regularization, format sensitivity, and sampled solution coverage.

Section 2 reviews related RLVR and reproducibility work. Section 3 describes the three optimization methods, Section 4 gives the experimental protocols, and Section 5 reports all experiment groups by purpose. Complete per-seed benchmark tables, exact prompts, response examples, and complete configuration-screen outcomes are provided in the appendices.

2. Related Work

Critic-free optimization and loss normalization. GRPO removes the learned critic used by PPO and estimates advantages from response groups [4]. Subsequent work has shown that implementation details are consequential: REINFORCE++ uses global advantage normalization [6], while the Dr. GRPO analysis identifies response-length bias induced by common GRPO normalization choices and proposes an alternative objective [7]. DAPO combines asymmetric clipping, dynamic sampling, token-level loss aggregation, and overlong-response handling [5]. These studies motivate our decision to treat comparisons between named methods as comparisons between complete training configurations unless sampling budget, aggregation, reward, and update exposure are matched.

Henderson et al. document seed- and environment-induced variance [26]; Engstrom et al. show that code-level optimizations materially change PPO and TRPO behavior [27]; and Agarwal et al. show that few-run aggregate comparisons can carry substantial statistical uncertainty [28]. This literature motivates our separation of benchmark-item uncertainty from training-run uncertainty and our refusal to infer an algorithm ranking from single selected checkpoints.

Sampling efficiency and collapse. The open-source TinyZero project used Countdown and multiplication as small-scale R1-Zero reproduction tasks [22], helping establish the setting audited here. Dynamic sampling excludes homogeneous reward groups so that retained prompts produce a learning signal, but it requires a variable number of rollouts [5]. Selective-rollout methods instead attempt to predict and skip uninformative prompts before generation [8]. A complementary line of work analyzes entropy collapse and proposes targeted clipping or KL control for high-covariance tokens [9]. Our study contributes a seed-level screen of a different observable failure mode—collapse to extremely short responses—and tests whether dynamic sampling, reward changes, overlong filtering, or KL regularization reliably prevent it.

Reward validity, elicitation, and transfer. Recent evidence cautions against equating benchmark improvement with newly acquired reasoning. Format, incorrect-label, and random rewards can elicit large gains in Qwen2.5-Math models, while the same signals are less reliable in other model families [10]. Large-k sampling studies similarly report that RLVR often improves pass@1 while failing to expand, and sometimes narrowing, the solution coverage of the base model [11]. Broader evaluations of math-tuned models find that transfer beyond the training domain is uneven and depends on the post-training method [12]. We connect these concerns by jointly reporting flexible scores, strict-format decomposition, held-out transfer, and pass@k after Countdown training.

3. Background: Optimization Methods

This section reviews the formulations of PPO, GRPO, and DAPO needed to distinguish the implemented methods; the empirical contribution begins in Section 5. Extended derivations are available in Ref. [13].

PPO [3] is widely used for RLHF, where its learned value function adds memory and optimization overhead [6]. GRPO [4] and DAPO [5] remove the critic in favor of group-relative advantage estimation. We summarize the three objectives to identify the method differences examined empirically.

3.1. Proximal Policy Optimization

PPO replaces TRPO's constrained second-order update [24] with a first-order clipped surrogate objective [3].

$$\mathcal{J}^{\mathrm{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\Big(r_t\hat{A}_t, \mathrm{clip}(r_t, 1-\epsilon, 1+\epsilon)\hat{A}_t\Big)\Big], \tag{1}$$

where $r_t = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\mathrm{old}}}(a_t|s_t)}$ is the likelihood ratio evaluated at the sampled state–action pair (s_t, a_t), $\hat{A}_t = \hat{A}^{\pi_{\theta_{\mathrm{old}}}}(s_t, a_t)$ is the estimated advantage (GAE for the displayed PPO configuration [25]), $\pi_{\theta_{\mathrm{old}}}$ is the pre-update policy, and π_θ is the updated policy. The first argument of min is the ordinary importance-weighted policy-gradient contribution; $\mathrm{clip}(r_t, 1-\epsilon, 1+\epsilon)$ truncates the ratio to the displayed interval, where ϵ is the symmetric clipping width.

Clipping and regularization. Clipping limits extreme likelihood-ratio contributions to the surrogate objective but does not enforce a hard trust-region constraint [15]. More broadly, empirical policy-gradient behavior can depart from the surrogate-based framework used to motivate it [14]. In our implementation, the PPO actor minimizes

$$L(\theta, \phi) = -\mathcal{J}^{\mathrm{CLIP}}(\theta) + c_1 L^{\mathrm{critic}}(\phi) - c_2\mathcal{H}(\pi_\theta), \tag{2}$$

where L^{critic} trains the value model and c_2 controls the optional entropy bonus. The matched PPO configuration sets both the entropy coefficient and the KL reward penalty

to zero. Thus, the reported comparison concerns complete implemented training configurations rather than abstract algorithms.

3.2. Group Relative Policy Optimization

For each prompt, GRPO samples G responses and replaces the critic with within-group relative advantages [4]:

$$\widehat{A}_i = \frac{R_i - \overline{R}}{\mathrm{std}(R_1, \ldots, R_G) + \epsilon_A}, \qquad \overline{R} = \frac{1}{G} \sum_{j=1}^{G} R_j. \tag{3}$$

A conventional GRPO presentation averages the clipped token objective within each response and then across the G responses. That response-normalized form is background and is not the aggregation used in the matched comparison. All matched group-relative runs use Verl's global `token-mean` aggregation. Defining the clipped token contribution as $\ell_{i,t}$, their implemented actor objective has the form

$$\mathcal{J}_{\mathrm{impl}}(\theta) = \mathbb{E}\left[\frac{1}{\sum_{i,t} m_{i,t}} \sum_{i,t} m_{i,t} (\ell_{i,t} - \beta k_{i,t}) \right], \tag{4}$$

where $\ell_{i,t} = \min\Big(r_{i,t}\widehat{A}_i, \mathrm{clip}(r_{i,t}, 1-\epsilon, 1+\epsilon)\widehat{A}_i \Big)$, $m_{i,t}$ masks padding and otherwise selects generated tokens, and $k_{i,t}$ is the Verl per-token low-variance KL estimator when KL loss is enabled. The matched GRPO and DAPO-style configurations set $\beta = 0$.

If every reward in a group is identical, Equation 3 is zero. Conversely, group standardization can amplify a small reward difference, including the 0.1 format credit used here. This supplies a plausible pressure toward reward-bearing shortcuts but is not evidence of a causal mechanism; the implementation-specific calculation is reported with the regularization diagnostics in Section 5.3.

3.3. Decoupled Clip and Dynamic Sampling Policy Optimization

The DAPO-style method uses the implemented token-mean objective in Equation 4 with asymmetric ratio clipping. The symmetric GRPO interval is replaced by $[1 - \epsilon_{\mathrm{low}}, 1 + \epsilon_{\mathrm{high}}]$. Published DAPO also uses dynamic sampling and overlong-response handling [5]; those components are disabled in the matched experiment so that the GRPO–DAPO-style contrast isolates the clipping range.

3.4. Comparison of Training Methods

PPO uses generalized advantage estimation and a learned critic; GRPO and DAPO use group-relative outcome advantages. Our matched comparison fixes data, reward, rollout count, actor updates, optimizer settings, aggregation, response cap, and horizon. PPO differs through its estimator and critic, while the GRPO–DAPO-style contrast isolates symmetric versus asymmetric clipping. We therefore compare the three implemented methods as a whole and do not claim that one component explains every difference.

4. Materials and Methods

We fine-tune Qwen2.5-1.5B-Instruct [16] on Countdown and evaluate on GSM8K [17], MATH [18], BBH [19], and MMLU-Pro [20].

4.1. Evaluation Protocol

Each benchmark is evaluated under two prompt conditions. The *standard* condition uses the benchmark-specific instruction; the *think* condition inserts the same item into the Countdown training wrapper and requests `<think>` and `<answer>` tags. Standard

prompts are the primary transfer condition. Think prompts test robustness to the training format. We report both conditions separately and never select the better prompt after observing the result. Exact prompt strings appear in Appendix A.

All main evaluations use vLLM greedy decoding (temperature 0), no repetition penalty, and at most 8,192 output tokens. Each model therefore produces one deterministic response per item. We score responses in two ways. The *strict* scorer accepts only an answer in the required final-answer format, using the last recognized final-answer marker. The *flexible* scorer accepts a correct strict-format answer and otherwise extracts an answer from completed, non-truncated text: the last number for GSM8K, symbolic matching for MATH, or the last answer choice for BBH and MMLU-Pro. Strict accuracy combines correctness with adherence to the required final-answer format; flexible accuracy is the primary approximation to problem correctness.

The complete test sets contain 1,319 GSM8K, 5,000 MATH, 6,511 BBH, and 12,032 MMLU-Pro items. BBH and MMLU-Pro use the stated custom zero-shot protocol rather than their conventional few-shot variants. Dataset revisions, chat wrapping, stopping rules, and scorer precedence are documented in Appendix A. A deterministic code-and-record audit corrected marker precedence and removed fallback credit from length-truncated responses; the archived outputs were then rescored with regression checks. The remaining limitation of the GSM8K last-number fallback is stated in the appendix.

Pass@k uses a different decoding protocol because it addresses sampled solution coverage rather than greedy pass@1 accuracy: temperature 0.6, top-$p = 0.95$, a 4,096-token cap, and 64 samples per MATH-500 item or 32 per GSM8K item. Its estimator and results are reported in Section 5.4.

4.2. Training Setup

All runs initialize from Qwen2.5-1.5B-Instruct and use Countdown as the sole training task [21]. The frozen scalar reward is 0.9 for a correct equation plus 0.1 for the complete reasoning-and-answer tag structure. A correct format-invalid response therefore scores 0.9, a fully formatted incorrect response scores 0.1, and a fully formatted correct response scores 1.0. Training uses Verl [23], bfloat16 weights, AdamW, one actor epoch per batch, and rollout temperature 1.0 on four NVIDIA RTX A6000 GPUs.

4.3. Matched Comparison of Training Methods

We froze the comparison protocol on 14 October 2026 before launching the nine runs. Each method uses the same training data, prompts, reward, software environment, paired data/rollout seeds (7, 8, and 9), and 300-step horizon. Each step samples 32 prompts with eight responses per prompt (256 trajectories) and performs eight actor updates with a response mini-batch of 32. The actor learning rate is 10^{-6} with a 10-step warmup and weight decay 0.1; KL loss, KL reward, and entropy regularization are disabled; loss aggregation is global token mean; and responses are capped at 4,096 tokens.

The methods differ only in their defining optimization components. PPO uses generalized advantage estimation ($\gamma = \lambda = 1$), a learned critic (learning rate 10^{-5}, mini-batch 32), and symmetric clipping 0.2/0.2. GRPO uses group-standardized advantages and symmetric clipping 0.2/0.2. The DAPO-style method uses the same group-relative estimator with asymmetric clipping 0.2/0.28. Dynamic sampling and overlong shaping are disabled, so the GRPO–DAPO-style contrast isolates clipping rather than every component of published DAPO. Actor-side exposure is matched; total compute is not identical because PPO also trains a critic.

Table 1. Matched comparison. Only the final three rows vary by method.

Setting	Value
Reward	0.9 correctness + 0.1 full-format credit
Prompts × responses per step	$32 \times 8 = 256$ trajectories
Actor mini-batch / updates per step	32 responses / 8
Actor LR / warmup / weight decay	10^{-6} / 10 steps / 0.1
KL loss / KL reward / entropy	off / off / 0
Aggregation / response cap	global token mean / 4,096
Seeds / horizon	paired 7, 8, 9 / 300 steps
PPO	GAE, critic, clip 0.2/0.2
GRPO	group-standardized advantage, clip 0.2/0.2
DAPO-style	as GRPO, clip 0.2/0.28, no dynamic sampling

The collapse endpoint was fixed before training: mean generated response length below 100 tokens over steps 90–100. Every matched run continued to step 300 regardless of this label, and every seed remained in the analysis. The step-300 checkpoints were evaluated on all four complete downstream test sets under both prompt conditions. With three seeds per method, means and sample standard deviations are descriptive. The single-seed KL sweeps and 22-run configuration screen use purpose-specific settings and are reported under their intended questions in Section 5.3; complete screen outcomes appear in Appendix D.

5. Results

5.1. Matched Training Outcomes

Table 2 reports every run in the matched experiment. The prespecified early-collapse rule labeled PPO seed 9, GRPO seed 9, and all three DAPO-style seeds. PPO seed 9 later lengthened again, but its step-300 Countdown accuracy was only 1.5%; the label therefore identifies an early trajectory, not an irreversible absorbing state. The other collapsed runs ended at 22–60 tokens and 24.8–27.1% Countdown accuracy. By contrast, the non-collapsed PPO and GRPO runs reached 52.0–59.1%.

Table 2. All nine matched runs. Mean tokens is computed over steps 90–100 and defines early collapse at fewer than 100 tokens. Countdown accuracy is measured at step 300. Downstream columns are each seed's mean flexible accuracy across GSM8K, MATH, BBH, and MMLU-Pro.

Method	Seed	Collapse	Mean tokens	Countdown (%)	Standard (%)	Think (%)
PPO	7	No	1374.1	55.7	49.2	46.2
PPO	8	No	447.0	53.5	49.4	47.8
PPO	9	Yes	16.3	1.5	48.5	36.2
GRPO	7	No	267.8	52.0	49.0	37.8
GRPO	8	No	725.0	59.1	49.6	42.3
GRPO	9	Yes	22.4	25.9	48.9	18.2
DAPO-style	7	Yes	55.6	25.8	48.6	18.4
DAPO-style	8	Yes	54.7	24.8	49.1	21.4
DAPO-style	9	Yes	51.0	27.1	49.0	20.2

The collapse counts are 1/3 for PPO, 1/3 for symmetric-clip GRPO, and 3/3 for asymmetric-clip DAPO-style training. The paired GRPO–DAPO-style contrast is underpowered (two-sided exact McNemar $p = 0.50$). This controlled contrast is consistent with lower stability under asymmetric clipping in the tested setting. Three seeds are too few to estimate population failure rates or to claim that asymmetric clipping alone is causal, especially because the PPO comparison also changes the estimator and adds a critic.

5.2. Prompt-Conditional Downstream Behavior

Under standard prompts, the fine-tuned models have nearly identical flexible accuracy to the base model: all three group means are within 0.2 percentage points of the base model's 49.0% four-benchmark mean (Table 3). Strict standard-prompt accuracy is 4.3–6.0 points higher than the base while flexible accuracy is unchanged, indicating better adherence to the required final-answer format rather than a broad correctness gain. MATH, BBH, and MMLU-Pro show no consistent standard-prompt improvement; the largest changes are concentrated on GSM8K, the benchmark closest to the arithmetic training domain. Complete per-seed benchmark values are in Appendix B.

Table 3. Downstream summary by training method. For each method, values are means across three independently fine-tuned model checkpoints, with sample standard deviation in parentheses. Each accuracy averages GSM8K, MATH, BBH, and MMLU-Pro with equal benchmark weight. Strict scoring requires the specified final-answer format; flexible scoring permits answer extraction from completed responses.

Method	Collapses	Strict standard	Flexible standard	Flexible think
Base model	–	39.2	49.0	41.6
PPO	1/3	44.8 (0.9)	49.0 (0.5)	43.4 (6.3)
GRPO	1/3	45.2 (0.9)	49.2 (0.4)	32.8 (12.8)
DAPO-style	3/3	43.4 (0.8)	48.9 (0.3)	20.0 (1.5)

The think condition exposes the failure hidden by standard prompts. Collapsed GRPO seed 9 averaged 18.2%, compared with 37.8% and 42.3% for the two non-collapsed GRPO seeds. The three DAPO-style seeds averaged 20.0%. PPO seed 9 was less severely affected but still fell to 36.2%, versus 46.2–47.8% for PPO seeds 7–8. The downstream effect of short-response collapse is therefore prompt-conditional. Under conventional benchmark prompts, collapsed checkpoints retain approximately base-model accuracy. However, when the identical questions are presented using the Countdown-style training wrapper, the checkpoints often revert to the learned short-answer behavior and accuracy drops sharply. In other words, the models still answered many problems correctly under standard benchmark prompts, but the Countdown-style training prompt triggered short responses and many errors.

5.3. Purpose-Specific Experiments and Robustness Checks

The evidence comprises several experiment groups designed to answer different questions. Table 4 reports them together while preserving the protocol associated with each question. The nine matched runs test differences among the three training methods under common actor-side exposure. The configuration screen tests the frequency and generality of short-response collapse. The KL sweeps test whether regularization preserves response length within the tested configurations. The prompt and pass@k controls test whether downstream score changes reflect correctness, adherence to the required final-answer format, or solution coverage. Because these protocols differ by design, their runs are not pooled as interchangeable replicates.

https://doi.org/10.3390/ai1010000

Table 4. Experimental groups, purposes, and appropriate interpretations.

Group	Evidence	Intended question	Interpretation
Matched comparison of training methods	Nine step-300 runs: PPO, GRPO, and DAPO-style; seeds 7–9	What changes when actor-side exposure, reward, data, optimizer settings, seeds, and horizon are matched?	Controlled method contrast
Configuration screen	22 runs plus a reconstructed legacy-configuration replay	How frequently does collapse occur across dynamic-sampling, no-DS, reward, KL, and environment variants?	Stability across tested configurations
Regularization diagnostics	Single-seed GRPO and DAPO-style KL sweeps	Can stronger KL preserve response length within a particular training configuration?	Configuration-specific diagnostic; no universal threshold
Evaluation controls	Prompt-only base-model control and pass@k evaluation of fixed checkpoints	Do score changes reflect adherence to the required final-answer format, probability redistribution, or expanded observed solution coverage?	Interpretation of downstream scores

Configuration screen and environment replay. In the matched comparison, early collapse occurred in 1/3 PPO runs, 1/3 GRPO runs, and 3/3 DAPO-style runs. The separate 22-run configuration screen tested dynamic sampling, matched-exposure no-DS, correctness-only reward with overlong filtering (A1), and A1 with KL 0.01. Sixteen runs met the same step-100 criterion: 5/6 with dynamic sampling, 3/6 with matched-exposure no-DS, 5/6 with A1, and 3/4 with A1 plus KL 0.01. All five current-environment dynamic-sampling runs collapsed near step 20, whereas a reconstructed legacy-configuration replay in the same environment remained healthy through step 100, averaging 1,527 tokens over steps 90–100 and reaching 47.2% validation accuracy. The replay rules out deterministic environment failure but does not isolate the responsible random stream or residual reconstruction difference. The complete run-level table appears in Appendix D.

Amplification of the format reward. For $G = 8$, seven rewards of 0 and one format-only reward of 0.1 give a mean of 0.0125. The Verl implementation uses the default Bessel-corrected sample standard deviation, $s \approx 0.0354$, so the format-only response receives advantage $(0.1 - 0.0125)/(0.0354 + \epsilon_A) \approx 2.47$ when ϵ_A is negligible. This calculation shows how a small format credit can become an order-one update signal; it does not identify the cause of collapse.

KL and response length. In a single-seed GRPO sweep with $G = 2$, KL coefficients at or below 0.005 collapsed to approximately 20-token responses within 12 steps, whereas coefficients 0.0075 and 0.01 recovered to approximately 1,000 tokens and coefficient 0.04 remained at 270–420 tokens. A separate DAPO-style $G = 2$ sweep also collapsed with KL off or 10^{-6} and survived with 10^{-4} or 0.0075. Because rewards, response limits, rollout budgets, and other settings differed, these values are configuration-specific diagnostics rather than recommended thresholds.

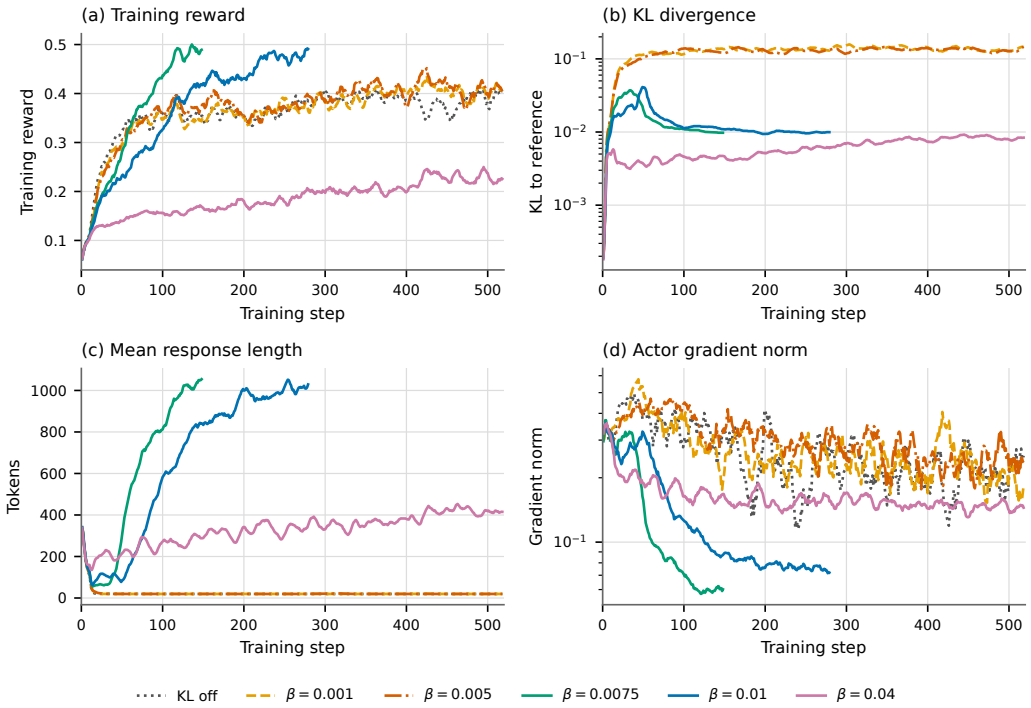


Figure 1. Single-seed GRPO diagnostics across KL-loss coefficients. Runs with $\beta \leq 0.005$ collapsed rapidly; stronger-KL runs retained or recovered response length. Each coefficient has one seed, so the figure supports a configuration-specific diagnostic rather than a general threshold.

5.4. Format and Coverage Controls

The prompt-only control isolates how much strict GSM8K accuracy can move without changing model weights. We kept the base model, all 1,319 GSM8K questions, chat template, greedy decoding, 8,192-token limit, and scorer fixed. The only intervention was an instruction requiring the response to end with exactly one separate final line of the form `#### N`; `N` had to be replaced by only the final numeric answer, and this mandatory line had to be the last line of the response. This addition increased use of the required final-answer format from 45.6% to 88.9% and strict accuracy from 33.4% to 58.5%, while flexible accuracy decreased from 72.3% to 67.1% (Table 5). Thus, a 25.2-point strict gain can be produced by presentation alone and need not indicate improved correctness.

Table 5. GSM8K prompt-only format control on the unchanged base model.

Condition	Strict accuracy (%)	Flexible accuracy (%)	Required-format rate (%)
Standard prompt	33.36	72.25	45.56
Mandatory-format prompt	58.53	67.10	88.93

The coverage control uses stochastic sampling for a different purpose than the greedy matched evaluation. For MATH-500, flexible pass@1 ranged from 54.1% to 55.2% and pass@64 from 90.8% to 92.2% across the base, healthy DS, no-DS, and strong-KL GRPO checkpoints. For GSM8K, pass@1 ranged from 70.4% to 73.9% and pass@32 from 96.3% to 97.1%. Differences visible at pass@1 largely narrowed at larger k. This pattern is consistent with probability redistribution among already sampled solutions, although finite-sample saturation does not prove equal latent capability.

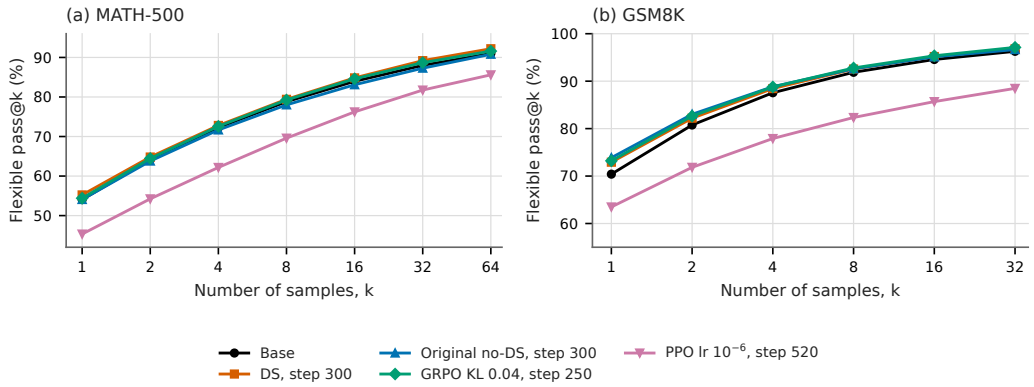


Figure 2. Flexible pass@k curves for fixed checkpoints on MATH-500 and GSM8K. Convergence at large k is descriptive and does not prove equal latent solution support.

6. Discussion

The matched comparison and the configuration studies answer different questions. Under the same reward, rollout count, actor-update count, optimizer settings, response cap, seeds, and 300-step horizon, early collapse occurred in 1/3 PPO runs, 1/3 symmetric-clip GRPO runs, and 3/3 asymmetric-clip DAPO-style runs. This controlled contrast is consistent with asymmetric clipping being less stable in this setting, but it does not establish a general ordering of PPO, GRPO, and DAPO: the sample contains only three seeds, PPO also changes the estimator and adds a critic, and published DAPO includes components deliberately disabled here. The configuration screen instead asks whether collapse recurs across dynamic-sampling, no-DS, reward, KL, and environment variants.

The downstream results explain why the failure can be missed. Under standard prompts, flexible four-benchmark accuracy is essentially identical for the base model and all three fine-tuned model groups. Under the training-format think prompt, collapsed checkpoints degrade sharply. The standard prompt therefore elicits residual capabilities that the think prompt suppresses, while the think prompt exposes the reward-compatible answer-only behavior learned during training. Neither condition alone is sufficient: standard-only evaluation understates the behavioral collapse, whereas think-only evaluation confounds model quality with an unusual wrapper that already changes the base model.

Strict scores provide a second warning. Matched training raises strict standard-prompt accuracy without raising flexible accuracy, and the prompt-only intervention raises strict GSM8K accuracy by 25.2 points while lowering flexible correctness. The coverage-control pass@k curves also converge at larger k. Together, these observations support probability and presentation changes more strongly than acquisition of broad new reasoning capability. The operational benefit of improved pass@1 or better adherence to a required output format can still be real, but it should not be described as general reasoning transfer.

The purpose-specific experiment groups provide complementary findings rather than a single pooled comparison. The 22-run configuration screen shows that short-response collapse occurs across dynamic-sampling, no-DS, reward, and KL variants. The reconstructed legacy-configuration replay shows that the current software environment does not deterministically cause the dynamic-sampling failures. The single-seed KL sweeps show that regularization can preserve response length in some configurations but do not identify a transferable safe coefficient. Together, these results reinforce configuration and run sensitivity without resolving the stochastic mechanism.

https://doi.org/10.3390/ai1010000

Several limitations remain. All experiments use one 1.5B-parameter model and one arithmetic training domain. Three seeds per matched method give only descriptive estimates, and no item-level significance test can substitute for more independent training runs. The collapse definition uses response length as an operational screen rather than a direct measure of reasoning quality. The think prompt is intentionally diagnostic and non-standard. BBH and MMLU-Pro use custom zero-shot prompts. Flexible GSM8K scoring relies partly on a last-number fallback whose independent human audit remains incomplete. Finally, actor exposure is matched, but PPO consumes additional compute for critic optimization.

Four reporting practices follow directly. First, publish every seed and use a predetermined checkpoint. Second, report flexible correctness beside strict scores. Third, evaluate both conventional prompts and the training-format prompt when a reward can create template-specific shortcuts. Fourth, treat response length, reward, and pass@1 as complementary diagnostics rather than interchangeable measures of reasoning.

7. Conclusion

In a matched nine-run Countdown experiment, PPO and symmetric-clip GRPO each collapsed in one of three seeds, while asymmetric-clip DAPO-style training collapsed in all three. Standard-prompt flexible accuracy remained at the base-model level for each fine-tuned model group, but think-prompt accuracy exposed large losses in the collapsed group-relative runs. Strict-score gains were largely attributable to adherence to the required final-answer format, as confirmed by a prompt-only control. The configuration screen and KL diagnostics show that the phenomenon is common but configuration-dependent. The central conclusion is therefore methodological: RLVR training methods should be compared with matched multi-seed runs, explicit collapse accounting, flexible scoring, and prompt-robust evaluation. These data do not justify a universal algorithm ranking, but they do show that standard single-prompt evaluation can conceal severe reward-compatible behavioral collapse.

Author Contributions: Conceptualization, Y.L.; methodology, Y.L.; software, Y.L.; validation, Y.L.; formal analysis, Y.L.; investigation, Y.L.; data curation, Y.L.; writing—original draft preparation, Y.L.; writing—review and editing, Y.L.; visualization, Y.L.; project administration, Y.L. The author has read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The public reproducibility artifact—including training and evaluation code, exact configurations, the run registry, compact derived results, vendored training and GSM8K parquet files, immutable revisions or file hashes for every dataset, environment locks, checkpoint hash manifests, and verification scripts—is available in the public GitHub repository. A DOI-bearing archived release will be cited in the final accepted version. Large model checkpoints, raw training logs, and per-item generation archives are available from the corresponding author on reasonable request where redistribution terms permit. Third-party source datasets and the Verl framework are available through their respective citations.

Acknowledgments: During preparation of this manuscript, the author used OpenAI Codex for language editing, manuscript restructuring, and code-assisted consistency checks. The author reviewed and edited all output and takes full responsibility for the content of this publication.

Conflicts of Interest: The author declares no conflicts of interest.

Appendix A. Evaluation Prompts, Dataset Identifiers, and Scoring

Chat wrapping and stopping.

Each benchmark prompt below was passed as the content of one user message to the Qwen tokenizer's `apply_chat_template` method with `add_generation_prompt=True`. Greedy evaluation used vLLM with bfloat16 weights, seed 0, temperature 0, and the model's default EOS stopping behavior; generation also stopped at the reported maximum-token limit. Pass@k used the same wrapping with seed 0, temperature 0.6, top-$p = 0.95$, and a 4,096-token cap. The current-environment ablations used Qwen model snapshot `989aa7980e4cf806f80c7fef2b1adb7bc71aa306`; the legacy DS/no-DS runs used a local model copy whose content hash was not recorded.

Checkpoint content identifiers. The machine-readable manifest at `eval/CHECKPOINT_MANIFEST.json` records all nine matched step-300 checkpoints and the four additional checkpoints used in purpose-specific diagnostic analyses. For each merged Hugging Face directory, it gives every relative file path, byte count, file SHA-256, and an aggregate SHA-256 over the canonical file list. Each directory contains nine files totaling 3,570,094,814 bytes. This permits an independently deposited checkpoint to be verified without printing 13 long identifiers in the article.

Exact prompt strings. Braced fields denote item-specific substitutions. The standard GSM8K user content was:

```
{question} Let's think step by step and output the final answer after "####".
```

The prompt-only format control appended the following text to that standard user content:

```
Your response must end with exactly one separate final line of the form
"#### N", where N is replaced by only the final numeric answer. This final
line is mandatory and must be the last line of your response.
```

The standard MATH user content was:

```
{problem}
Please reason step by step, and put your final answer within \boxed{}.
```

The standard BBH user content was:

```
{input}

Let's think step by step, and end your response with
"So the answer is <answer>."
```

The standard MMLU-Pro user content was:

```
The following is a multiple choice question about {category}. Think step by
step and then finish your answer with "the answer is (X)" where X is the
correct letter choice.

Question: {question}
Options:
{lettered options}
Answer: Let's think step by step.
```

The common think-condition string, including the wording and quotation marks inherited from the training wrapper, was passed as a *single user-message content string*:

```
system: You are a helpful assistant. You first thinks about the reasoning
process in the mind and then provides the user with the answer."
```

```
User: {question} Show your work in <think> </think> tags. And return the
final answer in <answer> </answer> tags, for example
<answer> {example} </answer>."
Assistant: Let me solve this step by step.
<think>
```

Dataset identifiers.

GSM8K used the 1,319-item test parquet distributed with the local Verl data preparation. Its upstream revision was not retained, but the exact evaluated file is vendored as `eval/data/gsm8k_test.parquet` with SHA-256 `f284e0ad015991d0059d265051a3c0915c5c1def15a8c01ac9a9142eeb87a180`. Exact comparison of every saved input/reference row with locally cached immutable snapshots recovered the other revisions: MATH used all seven test configurations from `EleutherAI/hendrycks_math` at `21a5633873b6a120296cce3e2df9d5550074f4a3` (5,000/5,000 rows); BBH used all 27 test configurations from `SaylorTwift/bbh` at `b5306be6f827cfafbb545ff5a51f96916029b0fd` (6,511/6,511 rows); and MMLU-Pro used the test split of `TIGER-Lab/MMLU-Pro` at `dd36ce4b34827164989f100331f82c5a29741747` (12,032/12,032 rows). The MATH-500 pass@*k* subset is an exact 500-item subset of that pinned MATH snapshot and is preserved as `eval/data/math500.jsonl`, SHA-256 `200806fb17234213e909649c09a14ce01ef836b32d3bcb1a261fcf96cda4470f`. The machine-readable manifest `eval/DATASET_MANIFEST.json` records canonical input digests, and `eval/verify_dataset_manifest.py` checks the archived files offline and can reconstruct the three revision-pinned datasets.

Scorer precedence. For GSM8K, strict scoring selects the last occurring explicit marker among `####`, `\boxed{}`, and `<answer>`. Flexible scoring accepts a correct strict marker; otherwise it uses the last numeric string only when the response did not end by length truncation. For MATH, strict scoring symbolically verifies the last explicit `\boxed{}` or `<answer>` content, while flexible scoring applies the same symbolic verifier to expressions parsed from a completed, non-length-truncated response. For BBH and MMLU-Pro, strict scoring selects the last recognized final-answer marker and flexible scoring otherwise selects the last parenthesized option letter only for completed, non-length-truncated responses; BBH additionally applies task-specific normalization for Dyck-language and word-sorting answers. The saved per-item files contain the raw response, extracted strict and flexible predictions, correctness flags, and finish reason so that extraction can be re-audited without regenerating model responses.

A planned independent audit of 40 GSM8K records, concentrated on base-model responses credited only by the last-number fallback, was not completed. We therefore report no human agreement or fallback-error estimate; flexible GSM8K accuracy should be interpreted as conditional on this deterministic extraction rule.

Appendix B. Complete Matched Benchmark Results

Tables A1 and A2 report every matched seed and both prompt conditions. These values underlie the four-benchmark means in Table 2; no seed or benchmark was omitted.

Table A1. Flexible accuracy (%) for the complete matched comparison. Std. denotes the benchmark-specific standard prompt; Think denotes the Countdown training wrapper.

	GSM8K		MATH		BBH		MMLU-Pro	
Model	Std.	Think	Std.	Think	Std.	Think	Std.	Think
Base	72.3	51.9	55.3	49.5	35.4	37.4	33.1	27.5
PPO s7	72.9	71.3	55.6	46.1	35.1	40.5	33.1	27.1
PPO s8	73.3	72.9	55.4	47.6	35.2	42.3	33.8	28.5
PPO s9	70.5	57.3	54.3	34.2	37.2	31.5	32.2	21.8
GRPO s7	73.2	55.6	55.0	39.4	34.4	37.7	33.4	18.6
GRPO s8	73.9	65.5	54.9	42.4	35.9	36.7	33.5	24.6
GRPO s9	73.1	7.1	54.6	11.1	34.2	34.5	33.7	20.1
DAPO-style s7	73.2	13.0	54.0	11.4	33.7	33.7	33.3	15.5
DAPO-style s8	73.4	8.3	54.6	15.5	34.5	36.1	33.9	25.6
DAPO-style s9	73.4	15.2	54.6	13.3	34.2	32.2	34.0	20.3

Table A2. Strict accuracy (%) for the complete matched comparison.

	GSM8K		MATH		BBH		MMLU-Pro	
Model	Std.	Think	Std.	Think	Std.	Think	Std.	Think
Base	33.4	48.4	55.1	47.4	35.2	36.2	33.1	27.5
PPO s7	59.4	71.2	55.5	44.2	34.9	40.4	33.1	27.1
PPO s8	51.9	72.7	55.4	47.0	34.7	42.0	33.8	28.5
PPO s9	55.5	56.4	54.1	32.3	37.1	31.5	32.2	21.8
GRPO s7	56.8	55.6	55.0	38.3	34.2	37.7	33.4	18.6
GRPO s8	53.8	65.5	54.9	41.7	35.7	36.7	33.5	24.5
GRPO s9	62.5	7.1	54.6	10.9	34.0	34.5	33.7	20.1
DAPO-style s7	56.6	12.8	53.9	10.8	33.5	33.7	33.3	15.5
DAPO-style s8	48.9	8.0	54.5	15.1	34.1	36.1	33.9	25.6
DAPO-style s9	50.1	15.1	54.6	12.5	33.9	32.2	34.0	20.3

Appendix C. Systematically Selected Collapse-Screen Outputs

Selection used metadata only, never response content. Among locally archived screen seeds 3–6, each configuration contributed the lowest-numbered endpoint-collapsed seed at its first sustained short-response event and, when available, the lowest-numbered endpoint-healthy seed at step 100. The recovered A1+KL seed is shown at both its earlier event and the endpoint. All seven selections happen to use the fixed first validation item (numbers 99, 26, 26; target 100). Exact unabridged responses, source-log paths, and the extraction script are provided in `ablation/representative_outputs.md`, `ablation/representative_outputs.json`, and `ablation/extract_representative_outputs.py`. Representative response excerpts are listed in Table A3.

Table A3. Content-blind examples from the collapse screen. "Endpoint" is the primary steps-90–100 label; the selected step is either the first sustained event or step 100. Character count is reported because the archived printed examples do not retain per-example token counts.

Configuration	Seed	Step	Chars	Score	Observation
Matched no-DS	5	30	132	0.1	Tagged but incorrect expression; endpoint collapsed.
Matched no-DS	3	100	12,252	0.0	Long unfinished search with no answer tag; endpoint healthy.
DS	3	20	33	0.0	Answer-only but incorrect expression; endpoint collapsed.
A1	3	25	33	0.0	Answer-only but incorrect expression; endpoint collapsed.
A1 + KL 0.01	3	30	31	0.0	Answer-only but incorrect expression; endpoint collapsed.
A1 + KL 0.01	4	25	31	0.0	Answer-only but incorrect expression during an early event.
A1 + KL 0.01	4	100	173	0.0	Longer explanation after recovery, but the equation remains incorrect; endpoint healthy.

Appendix D. Complete Collapse Screen

Table A4 reports every run in the fixed 22-run screen rather than only representative outcomes. Event step is the first sustained short-response window; a dash denotes right-censoring at step 100. The length and accuracy columns average available observations over steps 90–100.

Table A4. Complete 22-run collapse screen. The asterisk marks the legacy DS run whose stored random seeds are unknown; "1" is only a display identifier.

Configuration	Run/seed	Environment	Event step	Endpoint	Mean tokens	Validation accuracy (%)
DS	1*	Legacy	–	Healthy	1569.4	50.8
DS	2	Current	20	Collapsed	20.4	26.2
DS	3	Current	20	Collapsed	20.6	27.9
DS	4	Current	20	Collapsed	20.5	26.6
DS	5	Current	20	Collapsed	20.3	28.7
DS	6	Current	20	Collapsed	19.5	28.8
Matched no-DS	1	Current	45	Collapsed	61.9	22.4
Matched no-DS	2	Current	–	Healthy	229.6	18.8
Matched no-DS	3	Current	–	Healthy	1007.1	21.8
Matched no-DS	4	Current	–	Healthy	497.6	25.6
Matched no-DS	5	Current	30	Collapsed	41.7	20.3
Matched no-DS	6	Current	50	Collapsed	52.7	20.1
A1	1	Current	–	Healthy	2223.7	41.6
A1	2	Current	25	Collapsed	19.7	24.6
A1	3	Current	25	Collapsed	19.7	22.0
A1	4	Current	50	Collapsed	20.7	22.9
A1	5	Current	25	Collapsed	19.6	19.6
A1	6	Current	25	Collapsed	19.7	25.7
A1 + KL 0.01	3	Current	30	Collapsed	20.9	23.4
A1 + KL 0.01	4	Current	25	Healthy	132.2	25.3
A1 + KL 0.01	5	Current	30	Collapsed	18.5	22.8
A1 + KL 0.01	6	Current	30	Collapsed	20.7	23.5

References

1. OpenAI. Learning to Reason with LLMs. Available online: https://openai.com/index/learning-to-reason-with-llms/ (accessed on 12 October 2026).
2. DeepSeek-AI; Guo, D.; Yang, D.; Zhang, H.; Song, J.; Wang, P.; Zhu, Q.; Xu, R.; Zhang, R.; Ma, S.; Bi, X.; et al. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *Nature* **2025**, *645*, 633–638. https://doi.org/10.1038/s41586-025-09422-z.
3. Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; Klimov, O. Proximal Policy Optimization Algorithms. *arXiv* **2017**, arXiv:1707.06347.

4. Shao, Z.; Wang, P.; Zhu, Q.; Xu, R.; Song, J.; Bi, X.; Zhang, H.; Zhang, M.; Li, Y.K.; Wu, Y.; et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv* **2024**, arXiv:2402.03300.
5. Yu, Q.; Zhang, Z.; Zhu, R.; Yuan, Y.; Zuo, X.; Yue, Y.; Dai, W.; Fan, T.; Liu, G.; Liu, L.; et al. DAPO: An Open-Source LLM Reinforcement Learning System at Scale. *arXiv* **2025**, arXiv:2503.14476.
6. Hu, J.; Liu, J.K.; Xu, H.; Shen, W. REINFORCE++: Stabilizing Critic-Free Policy Optimization with Global Advantage Normalization. *arXiv* **2025**, arXiv:2501.03262.
7. Liu, Z.; Chen, C.; Li, W.; Qi, P.; Pang, T.; Du, C.; Lee, W.S.; Lin, M. Understanding R1-Zero-Like Training: A Critical Perspective. *arXiv* **2025**, arXiv:2503.20783.
8. Zheng, H.; Zhou, Y.; Bartoldson, B.R.; Kailkhura, B.; Lai, F.; Zhao, J.; Chen, B. Act Only When It Pays: Efficient Reinforcement Learning for LLM Reasoning via Selective Rollouts. *arXiv* **2025**, arXiv:2506.02177.
9. Cui, G.; Zhang, Y.; Chen, J.; Yuan, L.; Wang, Z.; Zuo, Y.; Li, H.; Fan, Y.; Chen, H.; Chen, W.; et al. The Entropy Mechanism of Reinforcement Learning for Reasoning Language Models. *arXiv* **2025**, arXiv:2505.22617.
10. Shao, R.; Li, S.S.; Xin, R.; Geng, S.; Wang, Y.; Oh, S.; Du, S.S.; Lambert, N.; Min, S.; Krishna, R.; et al. Spurious Rewards: Rethinking Training Signals in RLVR. *arXiv* **2025**, arXiv:2506.10947.
11. Yue, Y.; Chen, Z.; Lu, R.; Zhao, A.; Wang, Z.; Yue, Y.; Song, S.; Huang, G. Does Reinforcement Learning Really Incentivize Reasoning Capacity in LLMs Beyond the Base Model? *Adv. Neural Inf. Process. Syst.* **2025**, *38*. https://doi.org/10.52202/085713-1933; arXiv:2504.13837.
12. Huan, M.; Li, Y.; Zheng, T.; Xu, X.; Kim, S.; Du, M.; Poovendran, R.; Neubig, G.; Yue, X. Does Math Reasoning Improve General LLM Capabilities? Understanding Transferability of LLM Reasoning. *arXiv* **2025**, arXiv:2507.00432.
13. Lian, Y. Comparative Analysis and Parametric Tuning of PPO, GRPO, and DAPO for LLM Reasoning Enhancement. *arXiv* **2025**, arXiv:2512.07611.
14. Ilyas, A.; Engstrom, L.; Santurkar, S.; Tsipras, D.; Janoos, F.; Rudolph, L.; Madry, A. A Closer Look at Deep Policy Gradients. *arXiv* **2018**, arXiv:1811.02553.
15. Wang, Y.; He, H.; Tan, X. Truly Proximal Policy Optimization. *Proc. Mach. Learn. Res.* **2020**, *115*, 113–122.
16. Qwen Team. Qwen2.5: A Party of Foundation Models! Available online: https://qwenlm.github.io/blog/qwen2.5/ (accessed on 12 October 2026).
17. Cobbe, K.; Kosaraju, V.; Bavarian, M.; Chen, M.; Jun, H.; Kaiser, L.; Plappert, M.; Tworek, J.; Hilton, J.; Nakano, R.; et al. Training Verifiers to Solve Math Word Problems. *arXiv* **2021**, arXiv:2110.14168.
18. Hendrycks, D.; Burns, C.; Kadavath, S.; Arora, A.; Basart, S.; Tang, E.; Song, D.; Steinhardt, J. Measuring Mathematical Problem Solving with the MATH Dataset. *arXiv* **2021**, arXiv:2103.03874.
19. Suzgun, M.; Scales, N.; Schärli, N.; Gehrmann, S.; Tay, Y.; Chung, H.W.; Chowdhery, A.; Le, Q.V.; Chi, E.H.; Zhou, D.; et al. Challenging BIG-Bench Tasks and Whether Chain-of-Thought Can Solve Them. *arXiv* **2022**, arXiv:2210.09261.
20. Wang, Y.; Ma, X.; Zhang, G.; Ni, Y.; Chandra, A.; Guo, S.; Ren, W.; Arulraj, A.; He, X.; Jiang, Z.; et al. MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark. *arXiv* **2024**, arXiv:2406.01574.
21. Pan, J. Countdown-Tasks-3to4 Dataset. Available online: https://huggingface.co/datasets/Jiayi-Pan/Countdown-Tasks-3to4 (accessed on 21 February 2025).
22. Pan, J.; Zhang, J.; Wang, X.; Yuan, L.; Peng, H.; Suhr, A. TinyZero: Minimal Reproduction of DeepSeek R1-Zero. Available online: https://github.com/Jiayi-Pan/TinyZero (accessed on 12 October 2026).
23. Sheng, G.; Zhang, C.; Ye, Z.; Wu, X.; Zhang, W.; Zhang, R.; Peng, Y.; Lin, H.; Wu, C. HybridFlow: A Flexible and Efficient RLHF Framework. *arXiv* **2024**, arXiv:2409.19256.
24. Schulman, J.; Levine, S.; Abbeel, P.; Jordan, M.I.; Moritz, P. Trust Region Policy Optimization. *Proc. Mach. Learn. Res.* **2015**, *37*, 1889–1897.
25. Schulman, J.; Moritz, P.; Levine, S.; Jordan, M.I.; Abbeel, P. High-Dimensional Continuous Control Using Generalized Advantage Estimation. *arXiv* **2015**, arXiv:1506.02438.

26. Henderson, P.; Islam, R.; Bachman, P.; Pineau, J.; Precup, D.; Meger, D. Deep Reinforcement Learning That Matters. *Proc. AAAI Conf. Artif. Intell.* **2018**, *32*, 3207–3214. https://doi.org/10.1609/aaai.v32i1.11694.
27. Engstrom, L.; Ilyas, A.; Santurkar, S.; Tsipras, D.; Janoos, F.; Rudolph, L.; Madry, A. Implementation Matters in Deep Policy Gradients: A Case Study on PPO and TRPO. In *Proceedings of the International Conference on Learning Representations*; 2020.
28. Agarwal, R.; Schwarzer, M.; Castro, P.S.; Courville, A.C.; Bellemare, M.G. Deep Reinforcement Learning at the Edge of the Statistical Precipice. *Adv. Neural Inf. Process. Syst.* **2021**, *34*, 29304–29320.